

How the T20 Power Rankings Are Calculated

A power ranking answers a different question from the league table. The table asks **"who has earned the most points?"** The power ranking asks **"who is playing the best cricket right now?"** — so a team can sit fourth in the table and still top this list.

The founding rule: the maths decides the order, the AI does not. Every position is computed by fixed formulas from match data. The language model is only allowed to write the sentence explaining a team's position — it never sees, chooses, or influences the rank. Every ranking is reproducible, and every place is answerable with a number.

Part 1 — In plain English

Think of it as a **school report card**. Instead of one exam, each team is graded on nine subjects. Every subject is marked out of 100, some subjects count more than others, and the marks are combined into one final score. Sort by that score, and you have the rankings.

#	WHAT WE MEASURE	WHAT IT TELLS YOU	COUNTS FOR
1	Win %	How often they win. The foundation.	30%
2	Death overs (16–20)	The closing overs, where T20 games are decided: their hitting versus their bowling.	15%
3	Rolling Net Run Rate	Whether they out-score opponents, over the last 5 games only , so one early thrashing doesn't flatter them all season.	15%
4	Powerplay (overs 1–6)	Command of the opening burst — runs scored there minus runs conceded.	12%
5	Momentum (form)	Are they hot <i>right now</i> ? The latest match counts most.	12%
6	Home / away	Teams that only win at home are flattered; travelling well counts.	5%
7	Key players	A side missing its stars is not the same side, whatever last week's results say.	5%
8	Win quality	<i>How</i> they win and lose — a 60-run rout versus a last-ball escape.	4%
9	Strength of schedule	Whether the opponents faced were any good.	2%

Where the weights come from. They are set by cricket judgement, not by a training run. Win% leads because winning is the point. Net run rate and the two phase measures — the powerplay and the death overs — come next, because they capture *how* a side controls a game rather than just the result. Form, home/away, key players and win quality fill in the margins. They are deliberately round numbers, and they are published here so anyone can argue with them.

Why not fit them statistically? We tried. Weights tuned against simulated seasons scored better on the simulator, but the simulator built each team's powerplay and death-overs numbers straight out of a single hidden "strength" value — so those two measures were close to reading the answer key, and the fit handed them a third of the table. Real teams have phase *styles* that do not collapse to one number. Rather than publish a precision we cannot support, the weights are honest editorial choices, and we will re-fit only when a full season of real results exists to fit against.

Why momentum isn't simply "the last five". Recent matches count far more than old ones, fading smoothly rather than stopping dead at a cut-off. In practice the most recent match carries about 35% of this grade, and the last five together about 88%.

The score and the arrows. The nine graded subjects combine into a score out of 100; teams typically land between about **30 and 70**. Nobody scores 100 — that would mean leading every category at once. The **▲/▼ arrows** show movement against the previous published ranking: **▲2** means two places gained.

Nothing here is opinion. Most published power rankings are a writer's judgement call. This one is arithmetic: run the same matches through it twice and you get the same table, every time.

Part 2 — The technical method

Pipeline. `data` → `collect()` → nine metrics → normalise to `[0,1]` → `weighted sum × 100` → `sort`. Weights and bounds live in `weights.config.json`, never in code. Implementation: `src/engine.js`.

Notation. For team *t*: `W/P` = wins/played; `RF`, `OF` = runs scored and overs faced; `RA`, `OA` = runs and overs conceded. Every metric derives from raw per-innings data — nothing is hand-entered.

The nine components

1. Win percentage — `W / P`

2. Death-overs net (*overs 16–20*)

```
net = deathBatSR - deathBowLEcon × 16.67
```

The factor `100/6 = 16.67` converts bowling economy (runs/over) onto the batting strike-rate scale (runs per 100 balls), making the two dimensionally comparable.

3. Rolling NRR (*last 5 matches*)

$$rNRR = (\sum RF / \sum OF) - (\sum RA / \sum OA)$$

Per ICC convention a side **bowled out** is charged its full 20-over quota rather than the overs it actually faced, so being dismissed cheaply cannot flatter a run rate. A side that wins a chase early keeps the overs it used. Also reported: `trend = rolling - season`.

4. Powerplay dominance (*overs 1–6, i.e. 6 overs per innings per match*)

$$PP = (ppRunsFor / 6P) - (ppRunsAgainst / 6P) \quad [\text{runs per over}]$$

5. Momentum — exponentially weighted update over **every** match played, seeded at 0.5:

$$m_t = \alpha \cdot (\text{win} ? 1 : 0) + (1 - \alpha) \cdot m_{t-1}, \quad \alpha = 0.35$$

Match weights decay 0.350, 0.228, 0.148, 0.096, 0.063... — a **half-life of ≈1.6 matches**.

6. Home/away — $0.6 \cdot \text{awayWin\%} + 0.4 \cdot \text{homeWin\%}$, deliberately over-weighting away form.

7. Key-player availability — $\text{starsAvailable} / \text{squadStars}$, taken **as of the most recent match**.

8. Win quality — mean **signed runs margin over every match**, wins and losses alike:

defended	: own – opp	(runs)
chased	: (120 – balls used) × run rate	(runs equivalent)
lost	: the same quantity, negated	

Both outcomes are expressed in one unit — runs — so defending and chasing are directly comparable, and heavy defeats cost what heavy wins earn. Normalised over ±40 runs.

9. Strength of schedule — mean opponent win rate, **excluding their matches against this team**. Counting head-to-head made it a mirror of the team's own record (every win you take is a loss on an opponent's card), which inverted the metric. In a complete round robin every side then faces the identical field, so the value is the same for everyone and changes no ranking; it only carries information on an uneven schedule.

Normalisation and the final score

Each raw metric maps onto [0,1] via $\text{norm}(x) = \text{clamp}((x - \text{min}) / (\text{max} - \text{min}))$. Ratios (1, 6, 7, 9) are already 0–1; the rest use configured bounds — **death net ±60, rNRR ±2.00, powerplay ±3.00 rpo, win quality ±40 runs**. Then:

$$\text{score} = 100 \times \sum (w_k \cdot n_k)$$

weights 0.30, 0.15, 0.15, 0.12, 0.12, 0.05, 0.05, 0.04, 0.02, summing to exactly 1.00. **Ties** break on win% then rolling NRR. **Delta** = previous rank – current rank.

Choosing the weights. They are editorial, set by cricket judgement and stored in `weights.config.json` so they can be changed without touching the engine. An earlier set was fitted by maximising rank correlation against latent team strength over 300 synthetic seasons; that fit was withdrawn because the generator derived phase splits directly from strength, so it over-credited the powerplay and death-overs terms. The fitting method transfers unchanged to real data — only the target needs to become actual results.

Optional metrics (off by default)

Five extras can be enabled per session: **expected wins** (Pythagorean $RF^2 / (RF^2 + RA^2)$), **toss independence** (win% when *losing* the toss), **chase/set versatility**, **top-4 consistency**, and **bowling spread** (inverse share of wickets taken by the top two bowlers). Enabling them pushes total weight above 1.0, so scores scale up while the ordering stays valid.

Known limitations — stated deliberately

- **The weights are judgement, not evidence.** They are reasonable and openly published, but no live data yet proves this particular split is optimal. Re-fit against a full season of real results when one exists.
- **Correlated components.** Several measures move together — home/away tracks Win% very closely, and form overlaps rolling NRR — so the nine subjects are less independent than the table implies, and Win% effectively carries more than its stated 30%.
- **Key players** is a supplied figure, not yet a live ICC top-30 feed, so it reflects whatever availability was entered.
- **T20-shaped.** A 120-ball innings and a 6-over powerplay are assumed. Rain-shortened matches and other formats are not modelled, and the powerplay term assumes six powerplay overs per innings.
- **Strength of schedule is inert in a balanced league.** By design it is identical for every team in a complete round robin; it only does work on an uneven schedule.

Computed by `src/engine.js`; weights tunable in `weights.config.json` without code changes. Full fitting write-up in `parameters.md`. Every generated blurb is fact-checked against these numbers and logged with its sources for editorial review.